

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

MỞ RỘNG THIẾT KẾ MICROSERVICE

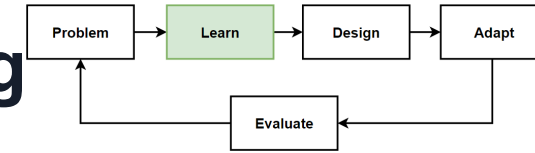
TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Scale the Microservices Architecture Design

Stateless and Stateful Application Horizontal Scaling

The Scale Cube

Learn: Scalability - Vertical Scaling - Horizontal Scaling



- **Scalability** is the **number of requests** an application can **handle**
- Measured by the number of requests and it can effectively support **simultaneously**.
- If no longer handle any more simultaneous requests, it has reached its **scalability limit**.
- To **prevent downtime**, and **reduce latency**, you must scale
- **Horizontal** scaling and **vertical** scaling both involve adding resources to your computing infrastructure
- **Horizontal scaling** by adding more machines
- **Vertical scaling** by adding more power

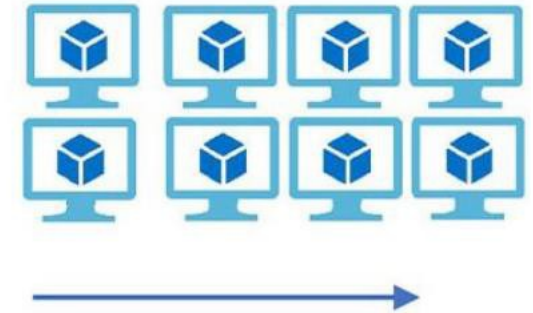
Vertical Scaling

(Increase size of instance (RAM , CPU etc.))



Horizontal Scaling

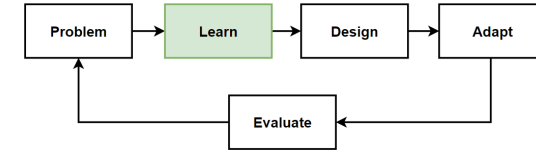
(Add more instances)



<https://www.webairy.com/horizontal-and-vertical-scaling/>

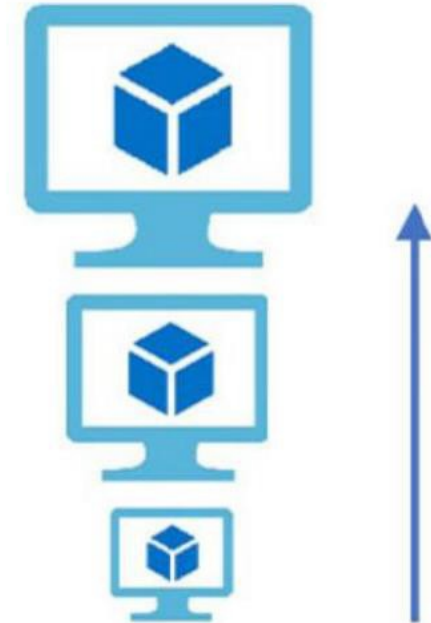
Vertical Scaling - Scale up

- **Vertical scaling** is basically makes the **nodes stronger**.
- Make the server stronger with **adding more hardware**. Adding more resources to a **single node**.
- Make **optimization** the hardware that will allow you to **handle more requests**.
- Vertical scaling keeps your existing infrastructure but **adding more computing power**.
- Your existing **code doesn't** need to **change**.
- Adding additional **CPU, RAM**, and **DISK** to cope with an increasing workload.
- By scaling up, you increase the capacity of a single machine. And it has limits. That is named **Scalability Limits**.
- Because even the hardware has **maximum capacity limitations**.
- For handling millions of request, we **need horizontal scaling** or scaling out.



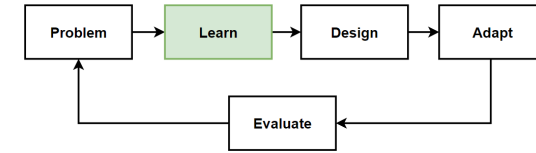
Vertical Scaling

(Increase size of instance (RAM , CPU etc.))



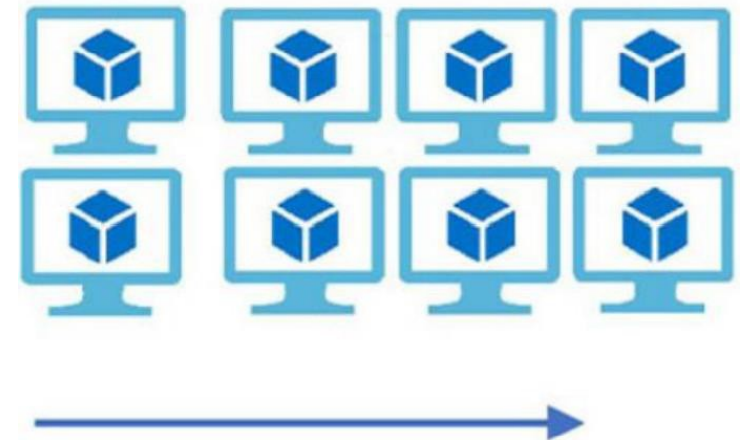
Horizontal Scaling - Scale out

- **Horizontal scaling** is **splitting** the load between **different servers**.
- Simply **adds more instances** of machines without changing to existing specifications.
- **Share** the **processing power and load balancing** across multiple machines.
- Horizontal scaling means **adding more machines** to the resource pool.
- **Scaling horizontally** gives you scalability but also **reliability**.
- Preferred way to scale in **distributed architectures**.
- When splitting into multiple servers, we need to consider if you have **a state or not**.
- if we have a state like database servers, than we need to manage more considerations like **CAP theorem**.

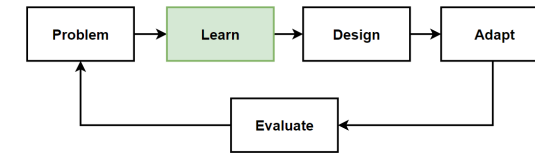


Horizontal Scaling

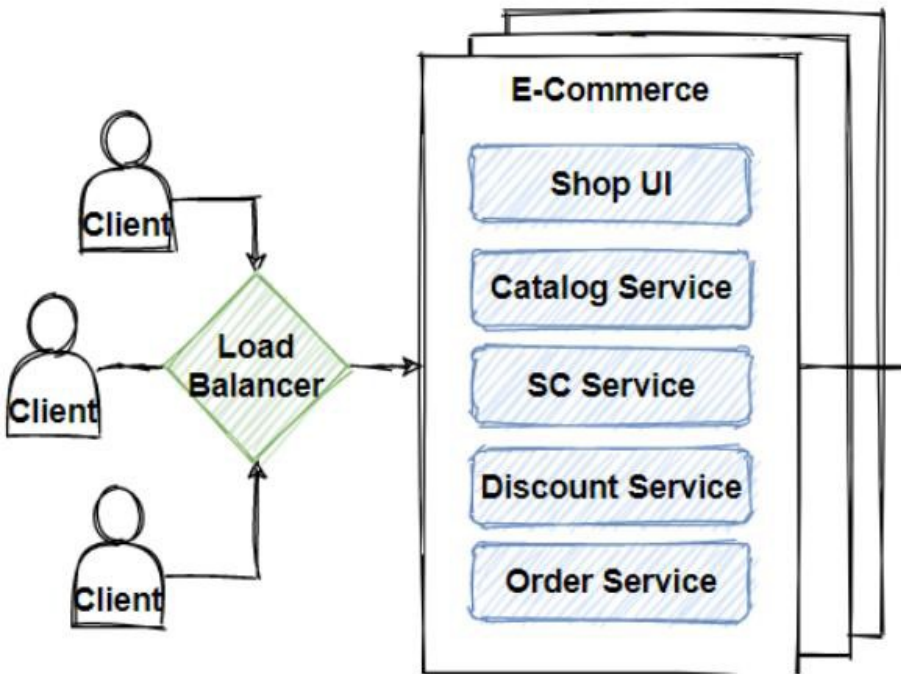
(Add more instances)



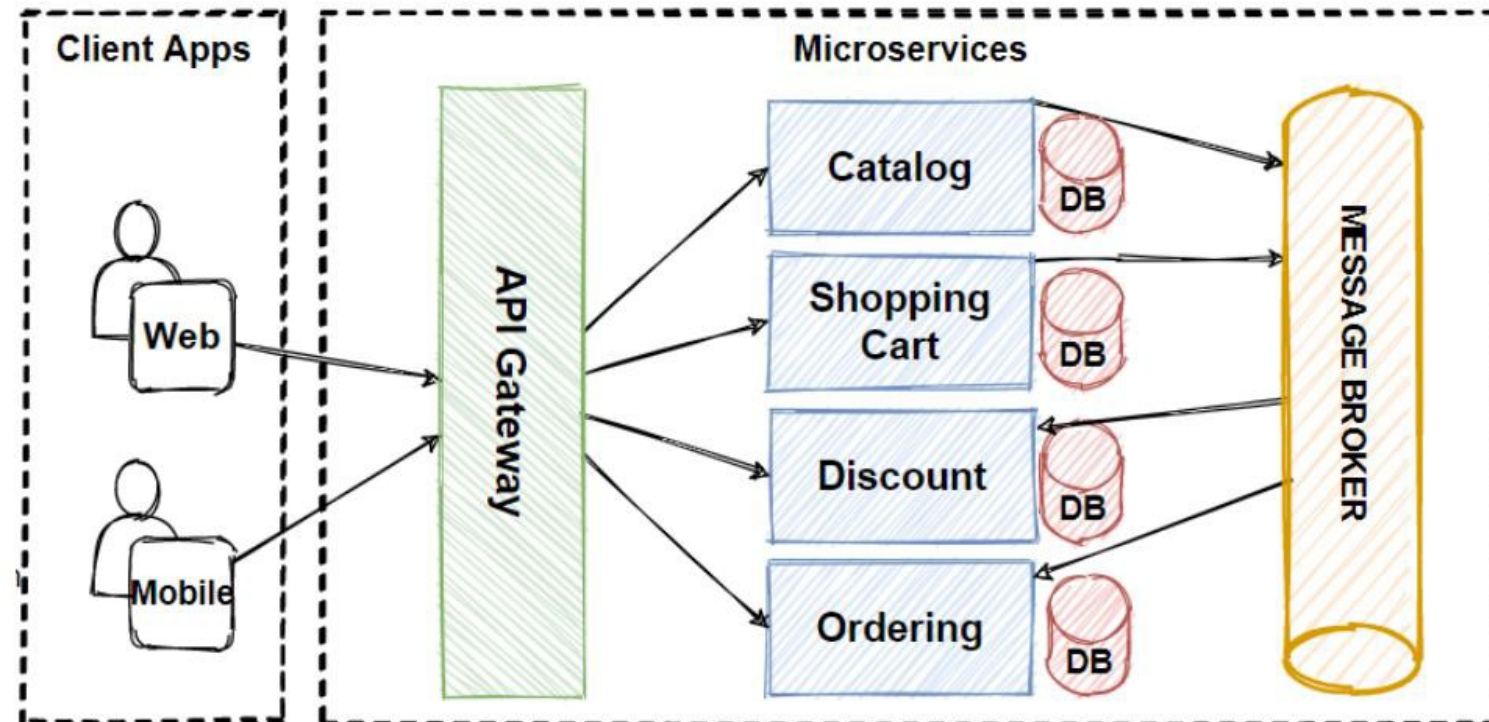
Scale the Microservices Architecture Design



Monolithic Architecture

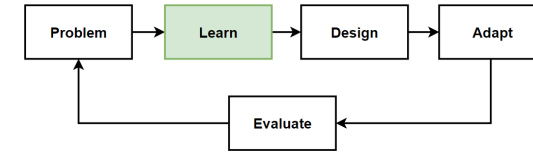


Microservices Architecture - Message Broker

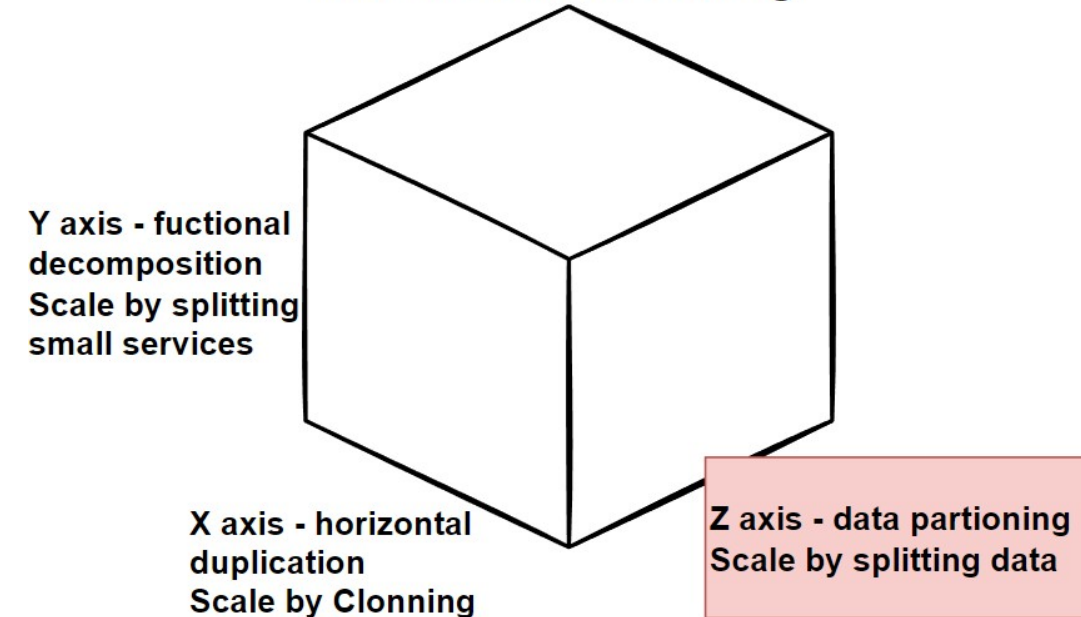


The Scale Cube

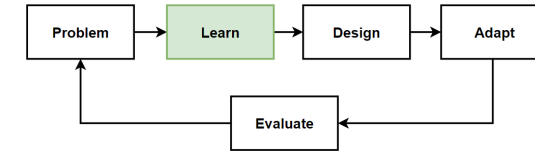
- **X-Axis:** Horizontal Duplication and Cloning of services and data
- **Y-Axis:** Functional Decomposition and Segmentation – Microservices
- **Z-Axis:** Service and Data Partitioning along Customer Boundaries - Shards/Pods
- **Functional decomposition** can be achieved by decoupling your architecture into functions **with Y-axis**.
- **Microservices** is an example for **functional decomposing**.
- **Y-axis** scaling splits the application into multiple, different services.
- **Combining** both **X** and **Y-Axis** scaling with microservices architecture can give **better scalability**.



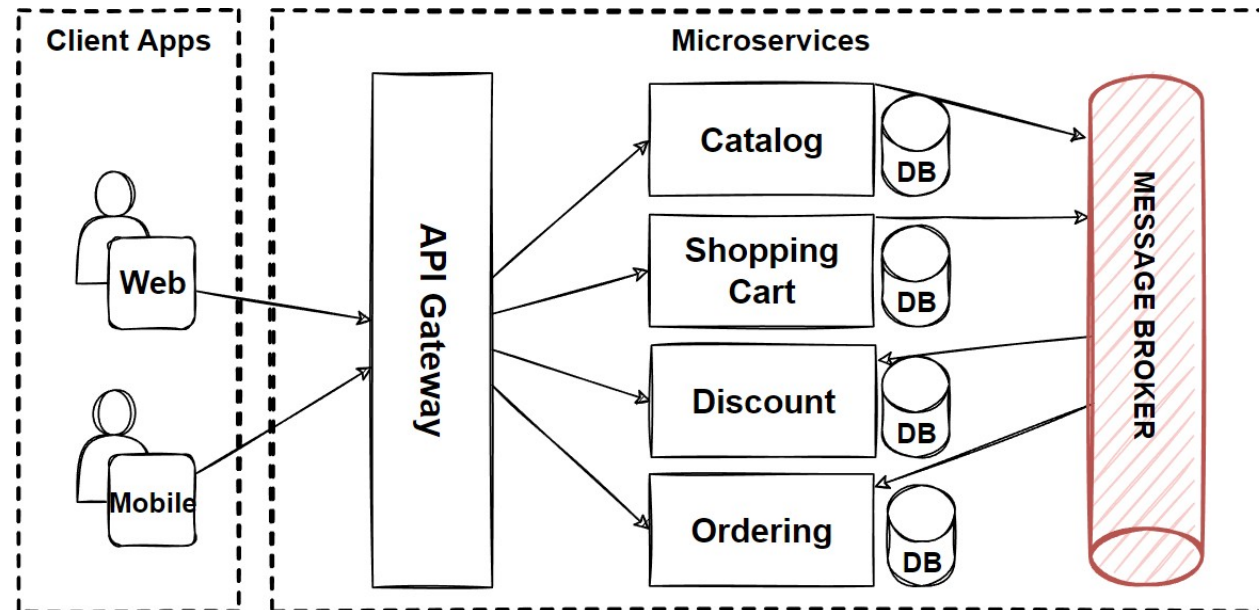
3 Dimensions of Scaling



Stateless and Stateful Application Horizontal Scaling

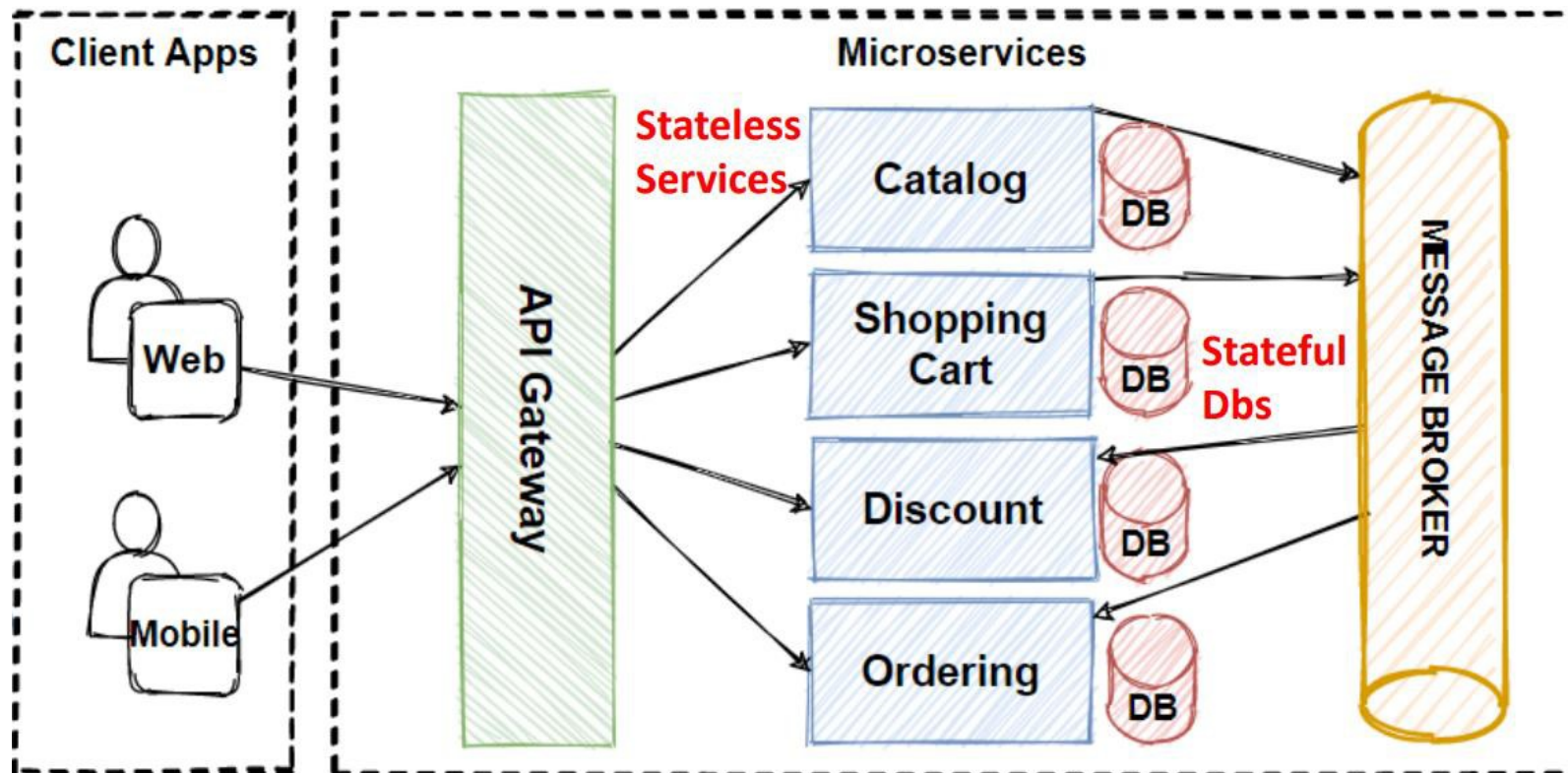
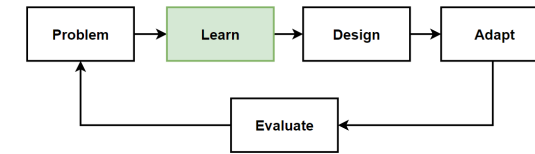


- **Scalability** of our architecture to **handle more** number of **concurrent requests**.
- **Vertical scaling** "scale up" is **increase** the **hardware** resources for existing servers.
- **Horizontal Scaling** "scaling out" is **splitting** the **load** between different servers.
- **If the server have a **state** or **not** ?**
- If the scaling server **don't have state**, just put the different servers and have a load balancer that direct the traffic.
- Orhestrator tools like **Kubernetes** have replica sets and we can increase to pod numbers easily.



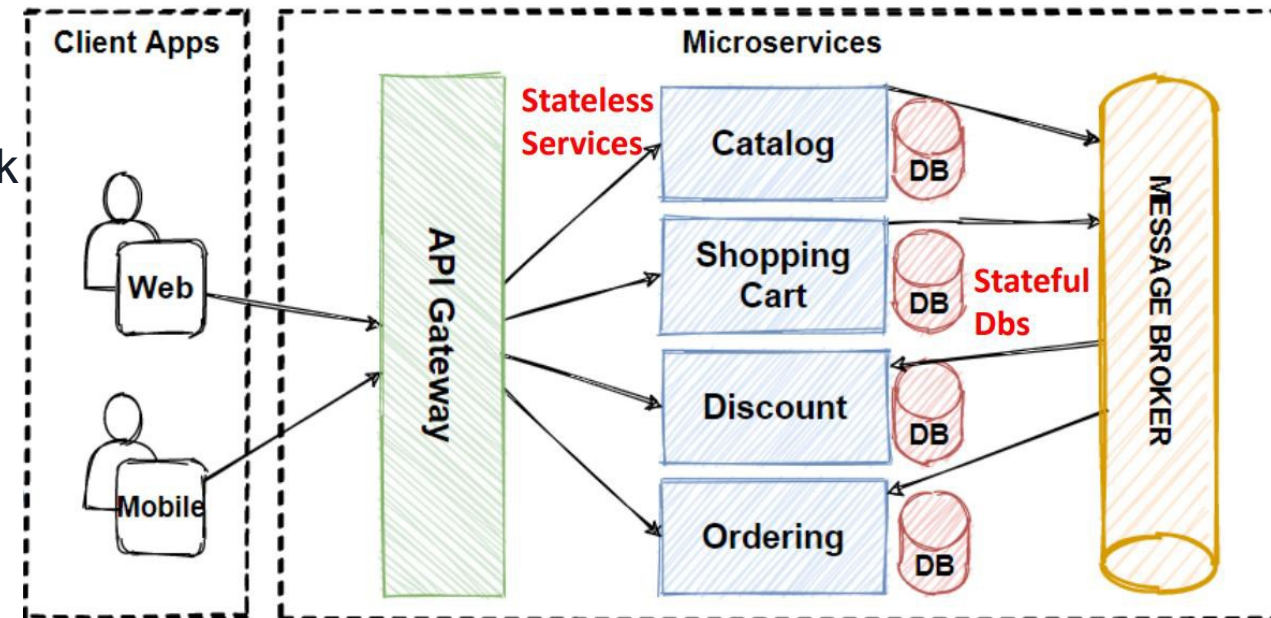
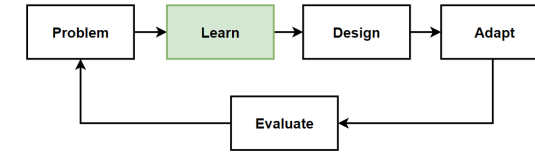
How to Scale Stateful Application Horizontal ?

- How can we **manage** this **stateful data** when **scaling** the application ?
- **CAP Theorem** to manage stateful server horizontal scaling operation.
- Microservices are "**stateless**" services that can scale easily.
- Databases have "**state**" and need to **consider CAP Theorem** when horizontally scaling for stateful servers.



2 type of Consistency Level

- What is the **consistency level requirements** of our microservices databases ?
- 2 type** of "consistency level":
- Strict Consistency**
When we save data, the data should affect and seen immediately for every client. Debit or withdraw on bank account.
- Eventual Consistency**
When we write any data, it will take some time for clients reading the data. Youtube video counters.
- When we horizontally scaling with the stateful database services, should **split the data** which is calling **sharding**.
- Database sharding** is also the way of **partitioning** and splitting database servers.
- How can we manage data in distributed systems ?



**CÁM ƠN ĐÃ CHÚ Ý
LẮNG NGHE!**